

```
// lib/main.dart
import 'dart:async';
import 'dart:math';
import 'dart:ui'; // For ImageFilter.blur
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:provider/provider.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_app_check/firebase_app_check.dart';
import 'package:flutter_dotenv/flutter_dotenv.dart';
import 'package:go_router/go_router.dart';
import 'package:flutter/foundation.dart' show kIsWeb, kDebugMode, kReleaseMode;
import 'package:url_launcher/url_launcher.dart' as url_launcher;
import 'package:simple_animations/simple_animations.dart';
```

```
// Import your project-specific files
import 'firebase_options.dart';
import 'models/user_account.dart';
import 'service/api_client.dart';
import 'state/user_provider.dart';
import 'state/proposal_provider.dart';
import 'state/goodwill_processing_provider.dart';
import 'state/ledger_provider.dart';
import 'pages/submit_goodwill_page.dart';
import 'pages/my_portfolio_page.dart';
import 'pages/governance_page.dart';
import 'pages/my_wallet_page.dart';
import 'pages/public_ledger_page.dart';
import 'screens/welcome_screen.dart';
import 'package:brightacts_frontend_app/models/tech_system.dart';
import 'screens/code_display_page.dart';
import 'screens/sign_up_screen.dart' as sign_up;
import 'screens/sign_in_screen.dart' as sign_in;
import 'state/auth_provider.dart' as MyAppAuthProvider;
import 'widgets/navigation_card.dart';
import 'widgets/dynamic_nebula_background.dart';
import 'app_state_providers.dart';
```

```
// --- Global Constants & Definitions ---
const String reCaptchaSiteKeyProd = String.fromEnvironment(
  'RECAPTCHA_SITE_KEY_PROD',
  defaultValue: "",
);
```

```
// --- MAIN ENTRY POINT ---
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await SystemChrome.setPreferredOrientations([DeviceOrientation.portraitUp,
DeviceOrientation.portraitDown]);
  await dotenv.load(fileName: ".env");
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  if (kDebugMode) {
    FirebaseAppCheck.instance.getToken(true).then((String? token) {
      if (token != null) {
        print('-----');
        print('Firebase App Check DEBUG TOKEN: $token');
        print('-> Add this to the Firebase Console (App Check -> Your App -> Manage debug
tokens)');
        print('-----');
      }
    });
  }
  final apiClient = PeoplesCoinApiClient();
  runApp(AppStateProviders(apiClient: apiClient, child: const BrightActsApp()));
}
```

```
// --- Theme Definition ---
class AppTheme {
  static ThemeData darkTheme = ThemeData(
    brightness: Brightness.dark,
    fontFamily: 'Roboto',
    primarySwatch: Colors.blue,
    scaffoldBackgroundColor: Colors.transparent,
    appBarTheme: const AppBarTheme(background-color: Colors.transparent, elevation: 0,
foregroundColor: Colors.white),
    textTheme: const TextTheme(
      bodyLarge: TextStyle(color: Colors.white70),
      bodyMedium: TextStyle(color: Colors.white70),
      headlineMedium: TextStyle(color: Colors.white),
      headlineSmall: TextStyle(color: Colors.white),
      titleLarge: TextStyle(color: Colors.white70),
    ),
    popupMenuTheme: PopupMenuThemeData(color: Colors.black.withOpacity(0.5), textStyle:
const TextStyle(color: Colors.white), shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10.0))),
    elevatedButtonTheme: ElevatedButtonThemeData(
      style: ElevatedButton.styleFrom(shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)), padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 12)),
```

```
    ),  
  );  
}
```

```
// --- HELPER CONSTANTS (MOVED UP FOR SCOPE) ---
```

```
class AppDurations {
```

```
  static const fast = Duration(milliseconds: 300);  
  static const medium = Duration(milliseconds: 400);  
}
```

```
class AppColors {
```

```
  static const governance = Color(0xFF8A2BE2);  
  static const portfolio = Color(0xFFCC6699);  
  static const recordAct = Color(0xFFDA70D6);  
  static const ledger = Color(0xFF00BFFF);  
  static const wallet = Color(0xFF6A5ACD);  
  static final buttonPrimary = Colors.amber[800];  
  static final translucentGovernance = AppColors.governance.withOpacity(0.2);  
  static final translucentPortfolio = AppColors.portfolio.withOpacity(0.2);  
  static final translucentRecordAct = AppColors.recordAct.withOpacity(0.2);  
  static final translucentLedger = AppColors.ledger.withOpacity(0.2);  
  static final translucentWallet = AppColors.wallet.withOpacity(0.2);  
}
```

```
class AppBreakpoints {
```

```
  static const double tablet = 768;  
  static const double desktop = 1200;  
}
```

```
// --- ROUTER CONFIGURATION ---
```

```
final _router = GoRouter(  
  initialLocation: '/',  
  routes: [  
    GoRoute(path: '/', builder: (context, state) => const AppStartupController()),  
    ShellRoute(  
      builder: (context, state, child) => AppShell(child: child),  
      routes: [  
        GoRoute(path: '/welcome', builder: (context, state) => const WelcomeScreen()),  
        GoRoute(path: '/sign_in', builder: (context, state) => const sign_in.SignInScreen()),  
        GoRoute(path: '/sign_up', builder: (context, state) => const sign_up.SignUpScreen()),  
        GoRoute(path: '/home', builder: (context, state) => const HomePage()),  
        GoRoute(path: '/code-viewer', builder: (context, state) {  
          final system = state.extra as TechSystem?;  
          if (system != null) { return CodeDisplayPage(title: 'Code Viewer', system: system); }  
          return const Scaffold(body: Center(child: Text('Error: No system data provided.')));  
        },  
      ),  
    ),  
  ],  
);
```

```

    ],
  ),
],
);
// --- APP SHELL, ROOT APP, and STARTUP CONTROLLER WIDGETS ---
class AppShell extends StatelessWidget {
  final Widget child;
  const AppShell({required this.child, super.key});
  @override
  Widget build(BuildContext context) {
    return Stack(
      children: [
        Positioned.fill(child: DynamicNebulaBackground()),
        child,
      ],
    );
  }
}

class BrightActsApp extends StatelessWidget {
  const BrightActsApp({super.key});
  @override
  Widget build(BuildContext context) {

SystemChrome.setSystemUIOverlayStyle(SystemUiOverlayStyle.light.copyWith(statusBarColor:
Colors.transparent, systemNavigationBarColor: Colors.black.withOpacity(0.5),
systemNavigationBarIconBrightness: Brightness.light));
    return MaterialApp.router(
      title: 'BrightActs',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.darkTheme,
      darkTheme: AppTheme.darkTheme,
      themeMode: ThemeMode.dark,
      routerConfig: _router,
    );
  }
}

class AppStartupController extends StatefulWidget {
  const AppStartupController({super.key});
  @override
  State<AppStartupController> createState() => _AppStartupControllerState();
}

class _AppStartupControllerState extends State<AppStartupController> {
  @override
  void initState() {

```

```

    super.initState();
    _initializeApp();
  }
  Future<void> _initializeApp() async {
    await WidgetsBinding.instance.endOfFrame;
    final authProvider = Provider.of<MyAppAuthProvider.AuthProvider>(context, listen: false);
    await authProvider.checkCurrentUser();
    if (mounted) {
      if (authProvider.user == null) {
        context.go('/welcome');
      } else {
        context.read<UserProvider>().fetchUser(authProvider.user!.uid);
        context.go('/home');
      }
    }
  }
}
@override
Widget build(BuildContext context) {
  return const Scaffold(backgroundColor: Colors.black, body: Center(child:
CircularProgressIndicator(color: Colors.deepPurple)));
}
}
// --- HOME PAGE WIDGET ---
class HomePage extends StatefulWidget {
  const HomePage({super.key});
  @override
  State<HomePage> createState() => _HomePageState();
}
class _HomePageState extends State<HomePage> {
  int _selectedIndex = 2;
  late final PageController _pageController;
  late final List<Widget> _pages;
  bool _showUiElements = true;
  @override
  void initState() {
    super.initState();
    _pageController = PageController(initialPage: _selectedIndex);
    _pages = _buildPages();
  }
  List<Widget> _buildPages() {
    return [
      _buildNavigationPage(title: 'Governance', description: 'Influence the future of the platform.
View active proposals, create new ones, and cast your votes on community-led initiatives.', icon:

```

```

Icons.gavel, cardColor: AppColors.translucentGovernance, buttonText: "View Proposals",
pageToOpen: const GovernancePage()),
  _buildNavigationPage(title: 'My Portfolio', description: 'Review your personal history of
contributions, track your impact, and see how your acts have strengthened the community.',
icon: Icons.account_balance_wallet, cardColor: AppColors.translucentPortfolio, buttonText:
"View My Acts", pageToOpen: const MyPortfolioPage()),
  _buildNavigationPage(title: 'Record Act', description: 'Document a new act of kindness or
contribution. Each verified act is rewarded with \'Loves\' and permanently added to the ledger.',
icon: Icons.favorite, cardColor: AppColors.translucentRecordAct, buttonText: "Submit a Bright
Act", pageToOpen: const SubmitGoodwillPage()),
  _buildNavigationPage(title: 'Public Ledger', description: 'Explore the transparent and
immutable record of every act of goodwill submitted by the community. A testament to our
collective impact.', icon: Icons.public, cardColor: AppColors.translucentLedger, buttonText:
"View Ledger", pageToOpen: const PublicLedgerPage()),
  _buildNavigationPage(title: 'My Wallet', description: 'Securely manage your \'Loves\'
balance, view transaction history, and send tokens to other members of the community.', icon:
Icons.wallet, cardColor: AppColors.translucentWallet, buttonText: "Open Wallet", pageToOpen:
const MyWalletPage()),
];
}
@override
void dispose() {
  _pageController.dispose();
  super.dispose();
}
Widget _buildNavigationPage({required String title, required String description, required
IconData icon, required Color cardColor, required String buttonText, required Widget
pageToOpen}) {
  return NavigationCard(title: title, description: description, icon: icon, cardColor: cardColor,
expandedContent: _cardContent(context, buttonText, pageToOpen: pageToOpen));
}
Future<void> _showSlidingFormOverlay(BuildContext context, Widget content) async {
  setState(() => _showUiElements = false);
  await Future.delayed(AppDurations.fast);
  await showGeneralDialog(
    context: context,
    barrierDismissible: true,
    barrierLabel: MaterialLocalizations.of(context).modalBarrierDismissLabel,
    barrierColor: Colors.black.withOpacity(0.5),
    transitionDuration: AppDurations.medium,
    transitionBuilder: (context, animation, secondaryAnimation, child) {
      final curve = CurvedAnimation(parent: animation, curve: Curves.easeOutCubic);
      return SlideTransition(position: Tween<Offset>(begin: const Offset(0.0, -1.0), end:
Offset.zero).animate(curve), child: FadeTransition(opacity: animation, child: child));
    }
  );
}

```

```

    },
    pageBuilder: (context, animation, secondaryAnimation) {
      final mediaQuery = MediaQuery.of(context);
      final isDesktop = mediaQuery.size.width >= AppBreakpoints.desktop;
      return Align(
        alignment: Alignment.center,
        child: Container(
          width: mediaQuery.size.width * (isDesktop ? 0.4 : 0.9),
          height: mediaQuery.size.height * 0.85,
          margin: EdgeInsets.only(top: mediaQuery.padding.top + (AppBar().preferredSize.height /
2), bottom: mediaQuery.padding.bottom),
          decoration: BoxDecoration(color: Colors.white.withOpacity(0.15), borderRadius:
BorderRadius.circular(25.0), border: Border.all(color: Colors.white.withOpacity(0.2))),
          child: ClipRRect(borderRadius: BorderRadius.circular(25.0), child: content),
        ),
      );
    },
  );
  setState(() => _showUiElements = true);
}
Widget _cardContent(BuildContext context, String buttonText, {required Widget pageToOpen})
{
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const Text('Detailed information about this section.', style: TextStyle(color: Colors.white70,
fontSize: 14)),
      const Spacer(),
      Center(
        child: ElevatedButton(
          onPressed: () {
            HapticFeedback.lightImpact();
            _showSlidingFormOverlay(context, pageToOpen);
          },
          style: ElevatedButton.styleFrom(backgroundColor: AppColors.buttonPrimary,
foregroundColor: Colors.white, padding: const EdgeInsets.symmetric(horizontal: 24, vertical:
12)),
          child: Text(buttonText, style: const TextStyle(fontSize: 16)),
        ),
      ),
      const SizedBox(height: 10),
    ],
  );
}

```

```

void _onItemTapped(int index) {
  HapticFeedback.lightImpact();
  _pageController.animateToPage(index, duration: AppDurations.fast, curve: Curves.ease);
}
@override
Widget build(BuildContext context) {
  final screenWidth = MediaQuery.of(context).size.width;
  final verticalMargin = screenWidth >= AppBreakpoints.desktop ? 40.0 : 20.0;
  return Scaffold(
    backgroundColor: Colors.transparent,
    extendBodyBehindAppBar: true,
    extendBody: true,
    appBar: _buildAppBar(context),
    body: SafeArea(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          AnimatedSlide(offset: _showUiElements ? Offset.zero : const Offset(0, -1.5), duration:
AppDurations.fast, child: Padding(padding: const EdgeInsets.symmetric(horizontal: 16.0), child:
_buildWelcomeHeader()),
            SizedBox(height: verticalMargin),
            Expanded(child: _buildPageView()),
            SizedBox(height: verticalMargin),
          ],
        ),
      ),
      bottomNavigationBar: _buildBottomNavBar(),
    );
}
PreferredSizeWidget _buildAppBar(BuildContext context) {
  return PreferredSize(
    preferredSize: const Size.fromHeight(kToolbarHeight),
    child: AnimatedSlide(
      offset: _showUiElements ? Offset.zero : const Offset(0, -1.5),
      duration: AppDurations.fast,
      child: AppBar(
        title: const SizedBox.shrink(),
        backgroundColor: Colors.transparent,
        elevation: 0,
        actions: [
          IconButton(
            icon: const Icon(Icons.settings),
            onPressed: () {
              HapticFeedback.mediumImpact();
            }
          )
        ]
      )
    )
  );
}

```



```

        showModalBottomSheet(context: context, backgroundColor: Colors.transparent,
isScrollControlled: true, builder: (_) => const SettingsBottomSheet());
    },
  ),
],
),
),
);
}
Widget _buildWelcomeHeader() {
  return Consumer2<MyAppAuthProvider.AuthProvider, UserProvider>(
    builder: (context, authProvider, userProvider, _) {
      final userAccount = userProvider.currentUser;
      final authUser = authProvider.user;
      if (userProvider.hasError) {
        return const Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('// CONNECTION INTERRUPTED', style: TextStyle(color: Colors.redAccent,
fontSize: 24, fontFamily: 'monospace')),
            Text(' Could not load user data.', style: TextStyle(color: Colors.white70)),
          ],
        );
      }
      final welcomeText = authUser != null ? 'Welcome, ${authUser.uid}' : 'Connecting...';
      return Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          MatrixText(targetText: welcomeText, style:
Theme.of(context).textTheme.headlineMedium?.copyWith(fontSize: 24, fontFamily:
'monospace'), isLoading: userProvider.isLoading),
          const SizedBox(height: 8),
          Row(
            children: [
              Text('Your Balance: ', style:
Theme.of(context).textTheme.titleLarge?.copyWith(fontSize: 18)),
              if (userProvider.isLoading) Text("*****", style:
Theme.of(context).textTheme.titleLarge?.copyWith(fontSize: 18, color: Colors.white))
              else if (userAccount != null) SlotMachineBalance(balance:
userAccount.balance.toInt(), textStyle:
Theme.of(context).textTheme.titleLarge!.copyWith(fontSize: 18, color: Colors.amber,
fontWeight: FontWeight.bold)),
              Text(' Loves', style: Theme.of(context).textTheme.titleLarge?.copyWith(fontSize: 18)),
            ],
          ),
        ],
      );
    },
  );
}

```

```

    ),
  ],
);
},
);
}
Widget _buildPageView() {
  return AnimatedOpacity(opacity: _showUiElements ? 1 : 0, duration: AppDurations.fast, child:
  Padding(padding: const EdgeInsets.symmetric(horizontal: 16), child:
  PageView.builder(controller: _pageController, itemCount: _pages.length, itemBuilder: (context,
  index) => _pages[index], onPageChanged: (index) {
    if (_selectedIndex != index) {
      HapticFeedback.selectionClick();
      setState(() => _selectedIndex = index);
    }
  },
  ),
  ));
}
Widget _buildBottomNavBar() {
  return AnimatedSlide(
    offset: _showUiElements ? Offset.zero : const Offset(0, 2),
    duration: AppDurations.fast,
    child: BottomNavigationBar(
      items: const [
        BottomNavigationBarItem(icon: Icon(Icons.gavel), label: 'Governance'),
        BottomNavigationBarItem(icon: Icon(Icons.account_balance_wallet), label: 'Portfolio'),
        BottomNavigationBarItem(icon: Icon(Icons.favorite), label: 'Record'),
        BottomNavigationBarItem(icon: Icon(Icons.public), label: 'Ledger'),
        BottomNavigationBarItem(icon: Icon(Icons.wallet), label: 'Wallet'),
      ],
      currentIndex: _selectedIndex,
      selectedItemColor: AppColors.buttonPrimary,
      unselectedItemColor: Colors.white70,
      onTap: _onItemTapped,
      backgroundColor: Colors.black.withOpacity(0.5),
      type: BottomNavigationBarType.fixed,
      showUnselectedLabels: false,
      showSelectedLabels: true,
    ),
  );
}
}
// --- SettingsBottomSheet ---

```

```

class SettingsBottomSheet extends StatelessWidget {
  const SettingsBottomSheet({super.key});
  void _launchSupportEmail(BuildContext context) async {
    const String businessEmail = 'support@brightacts.com';
    const String subject = 'Bright Acts Support Request';
    const String body = '...';
    final Uri emailLaunchUri = Uri(scheme: 'mailto', path: businessEmail, query:
'subject=${Uri.encodeComponent(subject)}&body=${Uri.encodeComponent(body)}');
    try {
      await url_launcher.launchUrl(emailLaunchUri);
    } catch (e) {
      if (context.mounted) {
        ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content: Text('Could not
launch email app.')));
      }
    }
  }
  @override
  Widget build(BuildContext context) {
    return BackdropFilter(
      filter: ImageFilter.blur(sigmaX: 5, sigmaY: 5),
      child: Container(
        padding: const EdgeInsets.all(24),
        decoration: BoxDecoration(color: Colors.black.withOpacity(0.75), borderRadius: const
BorderRadius.only(topLeft: Radius.circular(20), topRight: Radius.circular(20)), border:
Border(top: BorderSide(color: Colors.white.withOpacity(0.2)))),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Container(width: 40, height: 5, decoration: BoxDecoration(color: Colors.grey[700],
borderRadius: BorderRadius.circular(12))),
            const SizedBox(height: 20),
            const Text('Settings & Info', style: TextStyle(color: Colors.white, fontSize: 22, fontWeight:
FontWeight.bold)),
            const SizedBox(height: 20),
            _buildLinkButton(context: context, text: 'Donate to Bright Acts', icon:
Icons.volunteer_activism, color: Colors.pinkAccent, url:
'https://buy.stripe.com/4gM3cv65RfBnbOue7i24000'),
            const SizedBox(height: 10),
            _buildLinkButton(context: context, text: 'Change Background', icon: Icons.wallpaper,
onPressed: null),
            const SizedBox(height: 10),
            _buildLinkButton(context: context, text: 'Follow on LinkedIn', icon: Icons.group_work,
color: Colors.lightBlueAccent, url: 'https://www.linkedin.com/company/the-people-s-coin/'),

```

```

        const SizedBox(height: 10),
        _buildLinkButton(context: context, text: 'Docs & Codebase', icon: Icons.code, url:
'https://github.com/DonMichaelFeeney/Brightacts'),
        const SizedBox(height: 10),
        _buildLinkButton(context: context, text: 'Get Support', icon: Icons.support_agent,
onPressed: () => _launchSupportEmail(context)),
        const SizedBox(height: 10),
        Divider(color: Colors.white.withOpacity(0.3)),
        const SizedBox(height: 10),
        TextButton.icon(
          icon: const Icon(Icons.logout, color: Colors.redAccent),
          label: const Text('Sign Out', style: TextStyle(color: Colors.redAccent, fontSize: 16)),
          onPressed: () async {
            HapticFeedback.heavyImpact();
            Navigator.of(context).pop();
            await context.read<MyAppAuthProvider.AuthProvider>().signOut();
            if (context.mounted) {
              context.go('/welcome');
            }
          },
        ),
        SizedBox(height: MediaQuery.of(context).padding.bottom),
      ],
    ),
  ),
);
}

Widget _buildLinkButton({required BuildContext context, required String text, required
IconData icon, Color? color, String? url, VoidCallback? onPressed}) {
  final bool isEnabled = onPressed != null || url != null;
  final Color enabledColor = color ?? Colors.amber[700]!;
  final Color disabledColor = Colors.grey[600]!;
  return TextButton.icon(
    icon: Icon(icon, color: isEnabled ? enabledColor : disabledColor),
    label: Row(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Text(text, style: TextStyle(color: isEnabled ? Colors.white : Colors.grey[600], fontSize:
16)),
        if (!isEnabled) const Text(' (Coming Soon)', style: TextStyle(color: Colors.grey, fontSize:
12, fontStyle: FontStyle.italic)),
      ],
    ),
    onPressed: isEnabled ? (onPressed ?? () async {

```

```

    if (url != null) {
      final uri = Uri.parse(url);
      try {
        await url_launcher.launchUrl(uri, mode: url_launcher.LaunchMode.externalApplication);
      } catch (e) {
        if (context.mounted) {
          ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Could not
launch $url')));
        }
      }
    }
  } : null,
);
}
}
// --- NEW/ENHANCED WIDGETS AND HELPERS ---
class MatrixText extends StatefulWidget {
  final String targetText;
  final TextStyle? style;
  final bool isLoading;
  const MatrixText({super.key, required this.targetText, this.style, this.isLoading = false});
  @override
  State<MatrixText> createState() => _MatrixTextState();
}
class _MatrixTextState extends State<MatrixText> {
  static const String _chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#%&';
  final Random _random = Random();
  Timer? _timer;
  String _displayText = "";
  int _revealIndex = 0;
  @override
  void initState() {
    super.initState();
    _startAnimation();
  }
  @override
  void didUpdateWidget(covariant MatrixText oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (widget.targetText != oldWidget.targetText || widget.isLoading != oldWidget.isLoading) {
      _startAnimation();
    }
  }
  void _startAnimation() {
    _timer?.cancel();

```

```

    _revealIndex = 0;
    _displayText = _generateRandomString(widget.targetText.length);
    if (widget.isLoading) {
      _timer = Timer.periodic(const Duration(milliseconds: 60), (timer) {
        if (!mounted) { timer.cancel(); return; }
        setState(() { _displayText = _generateRandomString(widget.targetText.length); });
      });
    } else {
      _timer = Timer.periodic(const Duration(milliseconds: 40), (timer) {
        if (!mounted) { timer.cancel(); return; }
        setState(() {
          if (_revealIndex >= widget.targetText.length) {
            _displayText = widget.targetText;
            timer.cancel();
            return;
          }
          String scramblingPart = _generateRandomString(widget.targetText.length - _revealIndex);
          String revealedPart = widget.targetText.substring(0, _revealIndex);
          _displayText = revealedPart + scramblingPart;
          if (timer.tick % 2 == 0) { _revealIndex++; }
        });
      });
    }
  }
}

String _generateRandomString(int length) {
  if (length <= 0) return "";
  return String.fromCharCode(Iterable.generate(length, (_) =>
    _chars.codeUnitAt(_random.nextInt(_chars.length))));
}

@override
void dispose() {
  _timer?.cancel();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Text(_displayText, style: widget.style);
}
}

class SlotMachineBalance extends StatefulWidget {
  final int balance;
  final TextStyle textStyle;
  final Duration duration;

```

```

    const SlotMachineBalance({super.key, required this.balance, required this.textStyle,
this.duration = const Duration(milliseconds: 1500)});
    @override
    State<SlotMachineBalance> createState() => _SlotMachineBalanceState();
  }
  class _SlotMachineBalanceState extends State<SlotMachineBalance> with
SingleTickerProviderStateMixin {
    late AnimationController _controller;
    late Animation<double> _animation;
    int _displayBalance = 0;
    @override
    void initState() {
      super.initState();
      _controller = AnimationController(duration: widget.duration, vsync: this);
      _animateToBalance(widget.balance);
    }
    @override
    void didUpdateWidget(SlotMachineBalance oldWidget) {
      super.didUpdateWidget(oldWidget);
      if (widget.balance != oldWidget.balance) {
        _animateToBalance(widget.balance);
      }
    }
    void _animateToBalance(int newBalance) {
      _animation = Tween<double>(begin: _displayBalance.toDouble(), end:
newBalance.toDouble()).animate(CurvedAnimation(parent: _controller, curve:
Curves.easeOut)).addListener(() { setState(() { _displayBalance = _animation.value.toInt(); });
});
      _controller.reset();
      _controller.forward();
    }
    @override
    void dispose() {
      _controller.dispose();
      super.dispose();
    }
    @override
    Widget build(BuildContext context) {
      return Text(_displayBalance.toString(), style: widget.textStyle);
    }
  }
}

```